

Personal Finance Analyzer (WIP)

DELTAFLUX

1. Project Overview

DeltaFlux adalah aplikasi web fullstack yang membantu user mencatat transaksi keuangan sekaligus menganalisis pola pengeluaran dan pemasukan untuk mendeteksi potensi surplus atau defisit, rata-rata pengeluaran bulanan, dan analisa proyeksi kedepan.

Aplikasi ini bukan cuma pencatatan cashflow, tapi punya layer analisis berbasis rule sederhana seperti perbandingan rata-rata bulanan, lonjakan kategori tertentu, dan pola pengeluaran tidak konsisten.

Fokus MVP:

- Perbandingan periode (monthly)
- Deviation detection relatif terhadap rata-rata historis
- Perbandingan pertumbuhan expense vs pertumbuhan income
- Visualisasi tren

Goal utamanya: bikin sadar secara data-driven ke mana uang bocor, bukan cuma “feeling boros”.

2. Problem Statement

Problem 1:

Banyak orang merasa pengeluaran “kayaknya normal”, tapi sebenarnya ada kategori yang diam-diam naik perlahan setiap bulan tanpa disadari. Tanpa analisis berbasis data historis, kebocoran ini susah terlihat.

Problem 2:

Aplikasi pencatatan keuangan biasa cuma menampilkan summary statis (total income, total expense). Mereka jarang memberikan insight berbasis perbandingan waktu atau anomaly detection sederhana, sehingga user tetap harus mikir manual untuk menemukan pola.

3. Target User

Primary user: Jeremy Marcelino.

Karakteristik:

- Tech-savvy
- Mau insight, bukan cuma angka
- Nyaman dengan dashboard data
- Tertarik lihat pattern dan tren

Karena targetnya cuma satu user di awal, kita bisa:

- Fokus UX minimalis
- Ga perlu multi-user complexity
- Ga perlu role management
- Ga perlu scaling concern

Tapi desain schema tetap scalable kalau suatu hari mau multi-user.

4. Solution Overview

DeltaFlux adalah web application yang mencatat transaksi keuangan dan secara otomatis mendeteksi deviasi signifikan dalam pola pemasukan dan pengeluaran berdasarkan perbandingan historis dan pertumbuhan income.

Alih-alih menilai apakah pengeluaran “baik” atau “buruk”, sistem hanya mengidentifikasi perubahan pola yang tidak konsisten dengan baseline sebelumnya. Interpretasi akhir tetap berada di tangan user.

5. User Flow

a. Daily Usage Flow

User login → Masuk Dashboard → Klik “Add Transaction” → Isi form (type, amount, category, date, description) → Submit → Kembali ke Dashboard → Summary & chart otomatis ter-update

b. Analysis Flow

User login → Dashboard menampilkan:

- Total income periode aktif
- Total expense periode aktif
- Net cashflow
- Chart monthly
- Highlight kategori dengan deviasi signifikan

c. Optional Manual Insight Flow

User klik halaman “Insights” → Melihat perbandingan:

- Expense kategori X periode sekarang
- Rata-rata historis
- Persentase perubahan
- Perbandingan dengan pertumbuhan income

d. Data Review Flow

User buka halaman “Transactions”

→ Lihat list transaksi (paginated)

→ Filter by date range / category

→ Edit / delete transaksi jika perlu

6. MVP Features & Business Rules

6.1 Must-Have

No	Feature
1	User bisa register dan login
2	User bisa menambahkan transaksi (income / expense) dengan: – tanggal – deskripsi – jumlah – kategori – sumber (atm, gopay, dana, cash)
3	User bisa mengedit dan menghapus transaksi
4	User bisa melihat daftar transaksi berdasarkan bulan aktif
5	User bisa melihat ringkasan bulanan yang menampilkan: – total income bulan aktif – total expense bulan aktif – net cashflow
6	User bisa melihat total pengeluaran per kategori pada bulan aktif
7	User bisa melihat persentase perubahan pengeluaran tiap kategori dibanding bulan sebelumnya
8	Sistem menyorot kategori dengan deviasi signifikan (untuk penekanan visual)

6.2 Nice-to-Have

No	Feature
1	Sistem membandingkan deviasi dengan rata-rata 3 bulan terakhir
2	User bisa mengatur ambang deviasi untuk highlight
3	User bisa melihat riwayat insight tiap bulan

4	Dark mode
5	User bisa melihat analisis per minggu
6	Sistem menampilkan perbandingan pertumbuhan total income vs total expense
7	Sistem menampilkan grafik tren total pengeluaran beberapa bulan terakhir
8	Sistem menampilkan empty state insight jika data belum cukup

7. Analytics Approach (High-level Logic)

a. Baseline Period

Analisis dilakukan berdasarkan perbandingan bulan aktif dengan bulan sebelumnya (Month-over-Month / MoM).

Sistem tidak menggunakan moving average pada V1. Perbandingan hanya dilakukan terhadap satu periode sebelumnya untuk menjaga kesederhanaan logika dan implementasi.

b. Income Growth Calculation

Untuk bulan aktif:

- Hitung total income bulan ini
- Hitung total income bulan lalu
- Jika bulan lalu > 0 → hitung MoM %
- Jika bulan lalu = 0 → tampilkan status “Belum ada data untuk dibandingkan”

Formula MoM:

$$\text{MoM \%} = ((\text{Current} - \text{Previous}) / \text{Previous}) \times 100$$

c. Total Expense Growth Calculation

- Hitung total expense bulan ini
 - Hitung total expense bulan lalu
 - Terapkan logika MoM yang sama seperti income
-

d. Expense per Category Deviation

Untuk setiap kategori expense:

- Hitung total kategori bulan ini
- Hitung total kategori bulan lalu
- Jika bulan lalu $> 0 \rightarrow$ hitung MoM %
- Jika bulan lalu $= 0 \rightarrow$ tandai sebagai aktivitas baru / tidak dapat dibandingkan

Jika nilai absolut MoM % melebihi threshold internal (hardcoded 20%), sistem akan menyorot kategori tersebut sebagai perubahan signifikan.

Catatan:

Sistem hanya menampilkan perubahan secara kuantitatif dan tidak mengklasifikasikan perubahan sebagai “baik” atau “buruk”.

e. Edge Case Handling

- Jika bulan lalu tidak memiliki transaksi sama sekali $\rightarrow$ sistem tidak menampilkan analisis deviasi
- Jika kategori tidak memiliki nilai pada bulan lalu $\rightarrow$ sistem tidak menghitung MoM untuk kategori tersebut
- Nilai negatif tidak digunakan karena expense dan income dipisahkan berdasarkan tipe transaksi.

8. API Endpoint Plan

Dibangun dengan REST API

Method	Endpoint	Purpose
POST	/api/auth/register	Register user baru
POST	/api/auth/login	Login user & return JWT
GET	/api/categories	Ambil semua kategori milik user
POST	/api/categories	Tambah kategori baru
PUT	/api/categories/:id	Update kategori
DELETE	/api/categories/:id	Hapus kategori

POST	/api/transactions	Tambah transaksi
GET	/api/transactions?month=YYYY-MM	Ambil transaksi bulan tertentu
PUT	/api/transactions/:id	Edit transaksi
DELETE	/api/transactions/:id	Hapus transaksi
GET	/api/analytics/summary?month=YYYY-MM	Total income, expense, net + MoM
GET	/api/analytics/category-deviation?month=YYYY-MM	Total per kategori + MoM + highlight flag

Summary:

```
{
  income: number,
  incomeMoM: number | null,
  expense: number,
  expenseMoM: number | null,
  net: number
}
```

Category Deviation:

```
[
  {
    categoryId: string,
    categoryName: string,
    currentTotal: number,
    previousTotal: number,
    momPercentage: number | null,
    isSignificant: boolean
  }
]
```

9. Suggested Project Structure

9.1 Tech Stack

Stack final yang realistis dan beginner-friendly:

Backend

Node.js + TypeScript

Express

Prisma

PostgreSQL (hosted di Supabase)

Frontend (nanti)

React + TypeScript

Kalo udah settle pindah ke Next.js.

9.2 Data Schema

Users

field	type	notes
id	uuid	pk
email	varchar	unique
password_hash	varchar	
created_at	timestamp	
updated_at	timestamp	

Categories

field	type	notes
id	uuid	pk
user_id	uuid	fk → users.id
name	varchar	
type	enum('income','expense')	

created_at	timestamp	
------------	-----------	--

Transactions

field	type	notes
id	uuid	pk
user_id	uuid	fk
category_id	uuid	fk
type	enum('income','expense')	redundancy but intentional
amount	numeric(14,2)	jangan float
source	varchar	dropdown string (atm, gopay, cash, dll)
description	text	optional
transaction_date	date	bukan timestamp
created_at	timestamp	
updated_at	timestamp	

Indexing Plan:

1. Index Monthly Filtering (Core Analytics)

Digunakan untuk hampir seluruh query dashboard dan MoM analysis.

(user_id, transaction_date)

Tujuan: Mempercepat filtering transaksi berdasarkan user dan rentang tanggal (bulanan).

2. Index Category Deviation Analysis

Digunakan untuk perhitungan deviasi pengeluaran per kategori.

(user_id, category_id, transaction_date)

Tujuan: Mempercepat grouping dan agregasi transaksi berdasarkan kategori dalam periode tertentu.

3. Index Income vs Expense Summary (Optional Optimization)

Digunakan untuk perhitungan total income vs expense per bulan.

(user_id, type, transaction_date)

Tujuan: Mempercepat agregasi berdasarkan tipe transaksi (income/expense) dalam periode tertentu.

Index akan diimplementasikan melalui Prisma schema menggunakan @@index directive.

9.3 Folder Structure

```
src/
├── app.ts
├── server.ts
├── routes/
│   ├── auth.route.ts
│   ├── transaction.route.ts
│   └── analytics.route.ts
├── controllers/
│   ├── auth.controller.ts
│   ├── transaction.controller.ts
│   └── analytics.controller.ts
├── services/
│   ├── auth.service.ts
│   ├── transaction.service.ts
│   └── analytics.service.ts
├── repositories/ (optional tapi bagus)
│   └── transaction.repository.ts
├── middleware/
│   ├── auth.middleware.ts
│   └── error.middleware.ts
├── utils/
├── prisma/
│   └── schema.prisma
```

10. Roadmap

Phase 0 — Project Setup (Foundation)

Tujuan: Project bisa jalan, konek DB, dan struktur folder rapi.

Order	Task	Deliverable
1	Init backend project (Express + TypeScript)	Project folder dengan package.json & TS config.
2	Setup ESLint + Prettier	Standar coding yang konsisten (clean code).
3	Setup Prisma + PostgreSQL (local)	ORM siap pakai dan database terkoneksi.
4	Define .env	Keamanan kredensial & API keys.
5	Setup folder structure (modules, services, etc.)	Arsitektur backend yang modular dan scalable.
6	Initial migration (users, categories, transactions)	Schema DB yang merepresentasikan entitas utama.

Deliverable: Server running + database connected + migration sukses.

Phase 1 — Authentication Layer

Tujuan: User system dan keamanan route berfungsi.

Order	Task	Deliverable
1	Register & Login endpoint	API untuk registrasi dan autentikasi user.
2	Password hashing (bcrypt)	Data password user tersimpan aman (tidak plain text).
3	JWT Authentication	Token-based session management.
4	Auth middleware	Proteksi route agar hanya bisa diakses user yang login.

Deliverable: User bisa register, login, dan akses protected route.

Phase 2 — Core Transaction System (MVP Core)

Tujuan: Operasi CRUD transaksi stabil dan tervalidasi.

Order	Task	Deliverable
1	CRUD Transaction (Create, Read, Update, Delete)	Fungsi dasar pengelolaan data keuangan.
2	Get transactions with monthly filter	Fitur filter data berdasarkan periode bulan.
3	Category management (basic)	Pengelompokan transaksi (misal: Food, Transport).
4	Validation (Zod) & Error handling	Input data yang bersih dan pesan error yang standar.

Deliverable: User bisa input data dan ambil data bulanan.

Phase 3 — Analytics Engine (MVP Intelligence)

Tujuan: Kalkulasi angka MoM dan deviasi yang akurat.

Order	Task	Deliverable
1	Monthly summary endpoint (Income, Expense, Net)	Rekapitulasi total uang masuk dan keluar per bulan.
2	Income & Expense MoM growth % calculation	Data perbandingan performa keuangan antar bulan.
3	Category deviation logic	Identifikasi anomali di kategori tertentu berdasarkan data bulan lalu
4	Division by zero handling	Return valid error kalo data bulan lalu belum ada
5	Logic testing via Postman	Verifikasi kalkulasi angka sebelum masuk ke frontend.

Deliverable: Endpoint analytics mengembalikan angka yang konsisten.

Phase 4 — Basic Frontend Integration

Tujuan: Visualisasi data dan interaksi user melalui UI.

Order	Task	Deliverable
1	Setup Next.js & Auth flow integration	UI yang bisa login dan nyimpen session.
2	Transaction form & table	Input transaksi dan daftar riwayat yang user-friendly.
3	Dashboard summary cards & chart	Visualisasi angka agar mudah dipahami secara cepat.

Deliverable: User bisa lihat dashboard dan angka bergerak.

Phase 5 — Nice to Have

Tujuan: Peningkatan fitur dan UX.

Order	Task	Deliverable
1	Weekly analysis & Filtering	Breakdown data yang lebih detail (mingguan).
2	Dark mode & UI Polish	Tampilan aplikasi yang lebih modern dan nyaman.
4	Filter per kategori	User bisa memfilter income/expense per kategori
3	Configurable threshold	User bisa set batas limit budget sendiri.